BÀI TẬP 3 – TÌNH HUỐNG NGHIÊN CỨU (CASE STUDY)

1. Bối cảnh, mục tiêu và dữ liệu

Nhóm nghiên cứu cần tìm hiểu mối liên hệ giữa kết quả học tập, thời lượng tự học và mức độ hài lòng của sinh viên. Dữ liệu đến từ ba nguồn không đồng nhất. Mục tiêu ETL là trích xuất, chuẩn hóa, kết hợp có kiểm soát và cung cấp dữ liệu tin cậy để phân tích.

Nguồn	Trường/Thông tin	Chất lượng và cách sử dụng
CSV – bảng điểm	student_id; score (0–10)	Dữ liệu có cấu trúc; kiểm tra trùng, thiếu mã, sai thang điểm.
API – khảo sát	student_id; study_hours; satisfaction (1–5); submitted_at	Kiểm tra quyền truy cập, bản ghi lặp, thời điểm trả lời, thiên lệch mẫu.
PDF – báo cáo	Quy định điểm đạt 5,0/10; kỳ báo cáo	Trích văn bản và xác nhận thủ công; chỉ dùng làm tham chiếu, không ghép trực tiếp theo từng SV.

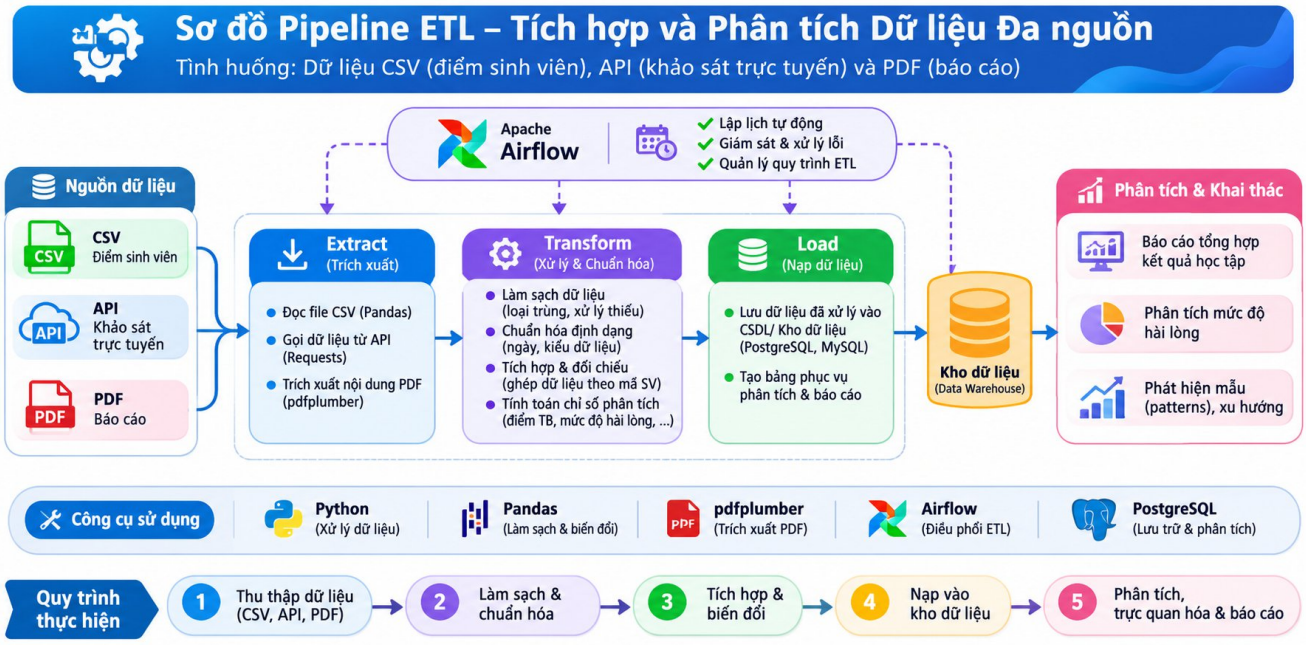
Đánh giá nguồn: CSV và API cần xác định đơn vị cung cấp, thời điểm cập nhật, tính đầy đủ và khả năng đối soát mã; PDF cần kiểm tra tác giả, năm học, phiên bản và độ chính xác trích xuất. Dữ liệu thật phải được ẩn danh trước khi đưa vào công cụ AI.

2. Lựa chọn công cụ

Phương án	Ưu điểm	Hạn chế / phù hợp
Pandas + Airflow	Linh hoạt, phân tích Python, DAG lập lịch, giám sát chạy lại [1], [2].	Cần viết mã, quản trị môi trường. Chọn cho nhóm nghiên cứu có Python.
Apache NiFi	Kéo-thả luồng dữ liệu, quản lý provenance và theo dõi dữ liệu [3].	Phù hợp luồng nhiều nguồn; cần cấu hình và vận hành dịch vụ.
Talend	Thiết kế ETL trực quan, nhiều connector.	Cần xem xét phiên bản, giấy phép và nguồn lực vận hành.

Quyết định: Dùng Pandas + Requests + pdfplumber + SQLite để chứng minh ETL tại Colab/máy cá nhân; dùng Airflow khi chạy định kỳ. pdfplumber phù hợp PDF có lớp văn bản; PDF scan cần OCR riêng [4].

3. Sơ đồ pipeline ETL (đề xuất)



Hình 1. Luồng ETL: CSV/API kết hợp theo student_id; PDF cung cấp quy tắc đối soát.

Nguyên tắc thiết kế: lưu bản gốc (raw) để truy vết; chỉ ghép dữ liệu có khóa student_id phù hợp; không tự động biến nội dung tổng hợp từ PDF thành bản ghi cá nhân. Tách bước kiểm tra dữ liệu (QA) trước khi phân tích.

4. Quy trình thực hiện tuần tự

Bước 1: Xác định nguồn và tiêu chuẩn dữ liệu

Đầu vào tối thiểu: diem_sinh_vien.csv; API /api/surveys có JSON dạng {"data": [...]}; bao_cao.pdf. Kiểm tra: quyền truy cập, mã sinh viên, mô tả thang điểm 0–10, mức hài lòng 1–5, giờ học 0–50/tuần và thời điểm khảo sát.

Prompt 01: Khảo sát nguồn

"Hãy lập data dictionary cho CSV điểm, JSON khảo sát qua API và PDF báo cáo; chỉ rõ khóa ghép student_id, miền giá trị hợp lệ, rủi ro nguồn và cách kiểm tra độ tin cậy. Không suy đoán trường chưa có."

Bước 2: Chuẩn bị môi trường thực thi

Công cụ: Google Colab hoặc Python 3.11+, Pandas, Requests, pdfplumber, Matplotlib, SQLite (tích hợp Python). Nếu tệp đặt trên Google Drive, mount Drive trước khi đọc.

```
!pip -q install pandas requests pdfplumber matplotlib
from google.colab import drive
drive.mount("/content/drive")
```

Tổ chức thư mục: /data/raw/ (CSV, bản JSON API, PDF) → /data/clean/ → /output/ (SQLite, biểu đồ và báo cáo). Không lưu API token vào mã nguồn.

Bước 3: Extract: lấy dữ liệu từ ba nguồn

Prompt 2 · Sinh lệnh Extract

"Viết mã Python đọc CSV bằng Pandas, gọi API bằng Requests với timeout và kiểm tra HTTP, trích văn bản từ PDF bằng pdfplumber. Giữ dữ liệu gốc; báo lỗi rõ nếu tệp thiếu hoặc API không phản hồi."

Bước 4: Lệnh Extract (tiếp)

```
import pandas as pd, requests, pdfplumber
score = pd.read_csv("diem_sinh_vien.csv", dtype={"student_id":
"string"})
resp = requests.get(API_URL, timeout=15, headers=HEADERS)
resp.raise_for_status()
survey = pd.DataFrame(resp.json()["data"])
with pdfplumber.open("bao_cao.pdf") as f:
    pdf_text = "\n".join(page.extract_text() or "" for page in f.pages)
```

API_URL và HEADERS là cấu hình của hệ thống khảo sát thật. Nếu chưa có API, có thể dùng JSON giả lập (ghi rõ mô phỏng); không gọi một URL không xác định. Sau khi trích PDF, đối chiếu thủ công nội dung "điểm đạt 5,0" trước khi áp dụng.

Bước 5: Transform: làm sạch và chuẩn hóa

```
score = score.drop_duplicates().copy()
score["student_id"] = score["student_id"].str.strip()
score["score"] = pd.to_numeric(score["score"], errors="coerce")
score = score[score.student_id.notna() & score.score.between(0, 10)]
survey["submitted_at"] = pd.to_datetime(survey["submitted_at"],
errors="coerce")
survey =
survey.sort_values("submitted_at").drop_duplicates("student_id",
keep="last")
for col in ["study_hours", "satisfaction"]:
    survey[col] = pd.to_numeric(survey[col], errors="coerce")
survey = survey[survey.study_hours.between(0, 50) &
survey.satisfaction.between(1, 5)]
```

Cần kiểm tra mã rỗng/trùng ở cả hai bảng và chuẩn hóa kiểu student_id trước khi ghép. Bản ghi lỗi được thống kê riêng; không tự động điền điểm hay hài lòng bằng 0.

Bước 6: Join và Load

```
result =
score.merge(survey[["student_id","study_hours","satisfaction"]],
    on="student_id", how="left", validate="one_to_one", indicator=True)
result["passed"] = result["score"] >= 5.0  # Ngưỡng từ PDF đã kiểm
chứng
print(result["_merge"].value_counts())
import sqlite3
with sqlite3.connect("research.db") as conn:

result.drop(columns="_merge").to_sql("student_etl",conn,if_exists="repl
ace",index=False)
```

LEFT JOIN giữ toàn bộ điểm hợp lệ, kể cả sinh viên chưa tham gia khảo sát. Chỉ số coverage = số bản ghi ghép được / số sinh viên có điểm hợp lệ. validate="one_to_one" tránh nhân bản bản ghi do trùng khóa [5].

Bước 7: Phân tích, trực quan hóa và QA

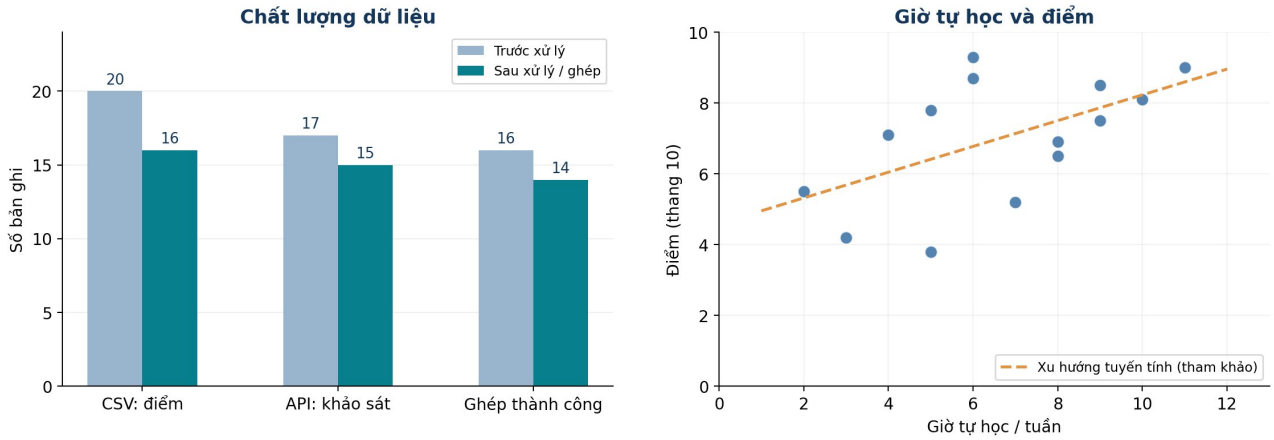
```
import matplotlib.pyplot as plt
result.dropna(subset=["study_hours"]).plot.scatter(
    x="study_hours", y="score", title="Giờ tự học và điểm")
plt.tight_layout(); plt.show()
```

QA tối thiểu: số dòng từng bước, số khóa thiếu, tỷ lệ ghép, tỷ lệ điểm hợp lệ, sự khác biệt giữa dữ liệu có/không tham gia khảo sát. Lưu biểu đồ và log chạy để báo cáo có thể tái lập.

5. Kết quả thực nghiệm minh họa

Đã chạy thử pipeline với bộ dữ liệu tổng hợp tự tạo, không chứa thông tin sinh viên thật. API được mô phỏng bằng phản hồi JSON; báo cáo PDF giả lập xác nhận ngưỡng điểm đạt = 5,0/10.

Chỉ số	Trước xử lý	Sau xử lý / diễn giải
Bảng điểm CSV	20 dòng	16 hợp lệ; loại 1 trùng, 3 dòng sai/thiếu.
Khảo sát API	17 phản hồi	15 hợp lệ; 1 bản lặp, 1 mức hài lòng ngoài thang.
Ghép CSV – API	—	14/16 sinh viên có khảo sát (87,5%).
Kết quả học tập	—	Điểm TB 6,82; tỷ lệ đạt 81,3% (ngưỡng 5,0).



Hình 2. Kiểm tra dữ liệu trước/sau làm sạch và mẫu phân tích giờ tự học – điểm.

Nhận diện mẫu (patterns) và diễn giải

- Thống kê mô tả: 81,3% sinh viên có điểm từ 5,0 trở lên trong mẫu 16 bản ghi; không suy rộng cho toàn trường.
- Trong 14 bản ghi ghép được, tương quan Spearman giữa số giờ tự học và điểm là $\rho \approx 0{,}506$ ($p \approx 0{,}065$): xu hướng dương mức vừa, nhưng chưa đủ bằng chứng ở mức ý nghĩa 5%.

- Điểm trung bình có vẻ tăng theo mức hài lòng, song mỗi nhóm chỉ có 2–4 người; dễ bị ảnh hưởng bởi mẫu nhỏ và sai lệch tự báo cáo.

Giới hạn: dữ liệu được tạo để minh họa quy trình nên không phải bằng chứng về hành vi học tập thực tế. Khảo sát có thể chịu thiên lệch chọn mẫu; tương quan không khẳng định quan hệ nhân quả.

6. Bộ prompt AI dùng lần lượt trong quy trình

Mỗi prompt chỉ hỗ trợ sinh hoặc đánh giá thao tác; người nghiên cứu chịu trách nhiệm xem nguồn, chạy mã và đối chiếu số liệu. Không tải dữ liệu cá nhân chưa ẩn danh lên dịch vụ AI.

PROMPT 01 · Đánh giá nguồn dữ liệu

> *"Đóng vai chuyên gia quản trị dữ liệu. Lập bảng đánh giá CSV điểm, API khảo sát và PDF báo cáo theo: độ tin cậy, mức cập nhật, đầy đủ, khóa ghép, rủi ro sai lệch. Chỉ sử dụng cấu trúc và metadata tôi cung cấp; đánh dấu phần chưa xác minh."*

PROMPT 02 · Tạo mã Extract

> *"Viết Python chạy Google Colab để đọc diem_sinh_vien.csv, lấy khảo sát từ API bằng requests (timeout, raise_for_status) và trích PDF bằng pdfplumber. Kiểm tra tệp/cột tồn tại, bảo vệ token và in số bản ghi từng nguồn."*

PROMPT 03 · Làm sạch và hợp nhất

> *"Dùng Pandas: xóa bản ghi trùng, chuẩn hóa student_id, chuyển score và study_hours về số, kiểm tra score 0–10 và satisfaction 1–5; chọn lần khảo sát mới nhất; ghép LEFT JOIN one_to_one; xuất bảng lỗi và tỷ lệ ghép, không tự thay dữ liệu thiếu bằng 0."*

PROMPT 04 · Phân tích và trực quan hóa

> *"Từ bảng đã ghép, tạo bảng số lượng trước/sau ETL, biểu đồ giờ tự học – điểm, tỷ lệ sinh viên đạt. Tính Spearman khi đủ dữ liệu; nêu 2–3 patterns dựa trên kết quả thực và cảnh báo mẫu nhỏ, thiếu phản hồi, không kết luận nhân quả."*

PROMPT 05 · Phản biện & kiểm tra

> *"Đóng vai giảng viên phản biện: kiểm tra mã có thể chạy, công thức tỷ lệ ghép, phạm vi PDF, rủi ro dữ liệu cá nhân, diễn giải biểu đồ và tính minh bạch của dữ liệu mô phỏng. Gợi ý chỉnh sửa theo thang điểm 25–25–25–15–10, không tự bịa nguồn hoặc số liệu."*

7. Tự động hóa khi triển khai định kỳ

Sau khi bản chạy thủ công ổn định, đóng gói các bước Extract → Transform → Validate → Load → Report thành các task trong Airflow DAG, lên lịch hàng ngày/tuần, đặt retry, lưu log và cảnh báo thất bại. Airflow quản lý thứ tự và giám sát, còn Pandas xử lý dữ liệu [1]. Không cần cài Airflow chỉ để chạy một lần trên Colab.

8. Tiêu chí nghiệm thu và đánh giá bài tập

Tiêu chí chấm	Minh chứng trong bài	Tỷ trọng
Tìm kiếm & đánh giá nguồn	Bảng 3 nguồn, rủi ro, thời điểm và nguồn tài liệu công cụ chính thức.	25%
Làm sạch & trực quan hóa	Quy tắc miền giá trị, số bản ghi loại, hình 2, khả năng tái lập.	25%
Phát hiện & phân tích patterns	Tỷ lệ đạt, tương quan, độ phủ ghép và giải thích giới hạn.	25%
AI/LLMs & tư duy phản biện	5 prompt và yêu cầu kiểm chứng kết quả AI, không bịa số liệu.	15%
Trình bày khoa học	Bố cục có sơ đồ pipeline, mục tiêu, kết quả, kết luận và nguồn.	10%

Checklist kiểm soát chất lượng

- Tệp và nguồn đọc được; API có cấu trúc JSON đúng, quyền truy cập và timeout; PDF trích được chữ hoặc có phương án OCR.
- Đã báo cáo số bản ghi mất/giữ, loại trùng, kiểm tra miền giá trị và tỷ lệ ghép; không bị tăng dòng ngoài dự kiến.
- PDF được đối chiếu nguyên văn trước khi áp dụng ngưỡng; nguồn PDF không có student_id thì không nối sai theo từng sinh viên.
- Bản xuất SQLite/CSV không chứa thông tin định danh trực tiếp; dữ liệu thực có sự đồng ý và quyền truy cập phù hợp.
- Biểu đồ có nhãn trục, đơn vị, chú thích mẫu; phần kết luận không đồng nhất tương quan với nhân quả.

Kết luận và đề xuất

Phương án Pandas + Airflow cân bằng khả năng xử lý linh hoạt, tính tái lập và nhu cầu tự động hóa của nhóm nghiên cứu. Quy trình ETL đa nguồn phải chú trọng chất lượng, nguồn gốc, khóa ghép và bảo vệ dữ liệu, không chỉ việc nối bảng. Với bài tập lớp học, có thể dùng Colab + SQLite để minh họa; khi có API và dữ liệu được phép sử dụng, thay nguồn mô phỏng bằng nguồn thực và thực hiện lại toàn bộ QA.

Tài liệu tham khảo (nguồn chính thức)

[1] Apache Airflow. What is Airflow? https://airflow.apache.org/docs/apache-airflow/stable/index.html

[2] pandas. read_csv API. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

[3] Apache NiFi. User Guide – Data Provenance. https://nifi.apache.org/nifi-docs/user-guide.html

[4] jsvine/pdfplumber. README (không tích hợp OCR). https://github.com/jsvine/pdfplumber/blob/stable/README.md

[5] pandas. merge API – validate/indicator. https://pandas.pydata.org/docs/reference/api/pandas.merge.html

[6] Requests. Quickstart – timeout/raise_for_status. https://requests.readthedocs.io/en/latest/user/quickstart/

Ghi chú: các số liệu minh họa được tạo và xử lý trong môi trường Python để kiểm tra pipeline; không trích dẫn như dữ liệu khảo sát thực tế.

BÀI TẬP 4: SO SÁNH VÀ LỰA CHỌN CÔNG CỤ

1. Mục tiêu và bối cảnh

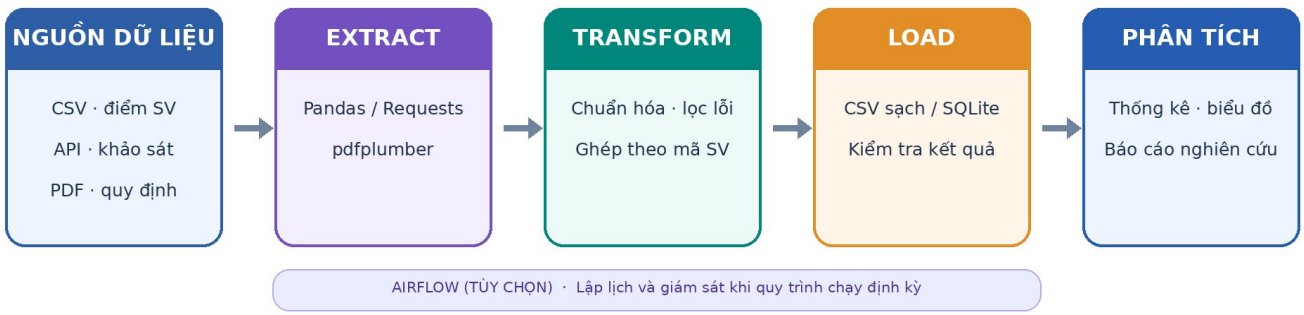
Nhóm nghiên cứu cần tổng hợp điểm sinh viên từ CSV, mức độ hài lòng từ API khảo sát và quy định đánh giá từ PDF. Mục tiêu là tạo bộ dữ liệu thống nhất, kiểm tra chất lượng và xây dựng báo cáo phân tích có thể tái lập.

2. Lựa chọn của nhóm

CHỌN PYTHON PANDAS: phù hợp nhất cho bước làm sạch, chuẩn hóa, ghép bảng và phân tích dữ liệu nghiên cứu quy mô nhỏ–vừa.

Đây là lựa chọn cho xử lý dữ liệu (wrangling), không phải hệ thống điều phối ETL toàn diện. Requests dùng gọi API; pdfplumber hỗ trợ đọc PDF; Airflow có thể bổ sung khi cần lập lịch và giám sát nhiều lần chạy.

3. Sơ đồ pipeline triển khai



Luồng xử lý: nhận ba nguồn → trích xuất → kiểm tra/làm sạch → ghép theo mã sinh viên → xuất bảng sạch và báo cáo. PDF được dùng để đối chiếu quy định; không ghép vào bảng cá nhân khi chỉ chứa thông tin tổng hợp.

4. Công cụ theo từng công đoạn

Công đoạn	Công cụ / lệnh tiêu biểu	Sản phẩm
Extract	pd.read_csv(), requests.get(), pdfplumber.open()	Bảng điểm, khảo sát, văn bản PDF
Transform	drop_duplicates(), to_numeric(), merge(validate=...)	Dữ liệu chuẩn hóa, có nhật ký lỗi
Load	to_csv(), to_sql() (SQLite tùy chọn)	Bộ dữ liệu phân tích
Analyze	groupby(), agg(), Matplotlib	Bảng tổng hợp, biểu đồ, nhận xét

5. Chức năng chính của Pandas

- Nhập/xuất: đọc CSV, Excel, JSON và ghi dữ liệu sạch; kết hợp Requests để lấy JSON từ API.
- Làm sạch: kiểm tra dữ liệu thiếu, trùng mã, sai kiểu, giá trị ngoài miền hợp lệ.
- Biến đổi & liên kết: chuẩn hóa mã sinh viên, merge/join giữa điểm và khảo sát, tính chỉ số tổng hợp.
- Phân tích: groupby, thống kê mô tả và chuẩn bị dữ liệu trực quan hóa.

6. Điểm mạnh và điểm yếu

Điểm mạnh	Giới hạn / cách khắc phục
Cú pháp ngắn, dễ kiểm thử và tái lập bằng notebook/script.	Cần kỹ năng Python; bổ sung hướng dẫn, kiểm thử, lưu mã nguồn.
Linh hoạt với bộ dữ liệu nhiều bảng, quy tắc làm sạch phức tạp.	Dữ liệu lớn có thể vượt bộ nhớ; cân nhắc SQL, Dask hoặc Spark.
Hệ sinh thái Python hỗ trợ PDF, API, biểu đồ.	Không tự quản lý lịch, retry và dashboard luồng; dùng Airflow khi cần.
Triển khai nhanh trên Colab hoặc máy cá nhân.	PDF quét có thể cần OCR; phải rà soát thủ công nội dung quy định.

7. So sánh với công cụ khác

Tiêu chí	Python Pandas	Apache NiFi	Apache Airflow
Bản chất	Thư viện xử lý dữ liệu	Nền tảng quản trị luồng dữ liệu	Nền tảng điều phối workflow
Thao tác	Viết mã Python	Thiết kế luồng trực quan	Viết DAG Python

Làm sạch / phân tích	Rất phù hợp cho thao tác bảng	Có processor, mở rộng khi cần	Gọi tác vụ xử lý bên ngoài
Luồng dữ liệu liên tục	Cần tự xây dựng thêm	Thế mạnh tích hợp, theo dõi luồng	Thiên về tác vụ batch có lịch
Case study hiện tại	Ưu tiên: dễ thực hành, phân tích	Tốt nếu thu thập nhiều luồng liên tục	Bổ sung khi tự động hóa định kỳ

Nhận định: Pandas tối ưu cho nghiên cứu mẫu nhỏ–vừa; NiFi phù hợp luồng thu thập liên tục; Airflow không thay thế Pandas mà điều phối các tác vụ ETL.

8. Ví dụ ứng dụng: đánh giá học tập & khảo sát sinh viên

Dữ liệu dưới đây là MÔ PHỎNG, không phải kết quả của một khảo sát thực tế. Mục tiêu minh họa cách ETL và giới hạn của diễn giải.

Nguồn	Cấu trúc ví dụ	Quy tắc kiểm tra
CSV điểm	ma_sv, diem	Mã không rỗng; điểm trong [0,10]
API khảo sát	ma_sv, hai_long	Mã duy nhất; hài lòng trong [1,5]
PDF quy định	Ngưỡng đạt: điểm ≥ 5 (giả định)	Kiểm tra thủ công phiên bản, ngữ cảnh quy định

Quy trình thao tác tuần tự

Bước	Thực hiện / lệnh	Kiểm chứng
01. Khảo sát	read_csv(), requests.get(), extract_text()	Tên cột, số dòng, nguồn và thời điểm
02. Làm sạch	drop_duplicates(), to_numeric(), between()	Số bản ghi loại; giá trị bất hợp lệ
03. Tích hợp	merge(on="ma_sv", validate="one_to_one")	Tỷ lệ ghép; mã không đối ứng
04. Xuất dữ liệu	to_csv(..., encoding="utf-8-sig")	Đọc lại tệp, đối soát số dòng
05. Phân tích	mean(), value_counts(), biểu đồ	Không suy diễn từ mẫu quá nhỏ

Mã Python minh họa (Colab/Jupyter)

```
import pandas as pd
import requests
```

```
import pdfplumber

diem = pd.read_csv("diem_sinh_vien.csv", dtype={"ma_sv":"string"})
r = requests.get(API_URL, timeout=15); r.raise_for_status()
khao_sat = pd.DataFrame(r.json()["data"])
with pdfplumber.open("bao_cao.pdf") as f:
    quy_dinh = "\n".join(p.extract_text() or "" for p in f.pages)
# Quy định phải được người nghiên cứu kiểm chứng thủ công.
diem["diem"] = pd.to_numeric(diem["diem"], errors="coerce")
diem = diem.dropna(subset=["ma_sv"]).drop_duplicates("ma_sv")
diem = diem[diem["diem"].between(0, 10)]
khao_sat = khao_sat.dropna(subset=["ma_sv"]).drop_duplicates("ma_sv")
khao_sat["hai_long"] = pd.to_numeric(khao_sat["hai_long"],
errors="coerce")
khao_sat = khao_sat[khao_sat["hai_long"].between(1, 5)]
ket_qua = diem.merge(khao_sat[["ma_sv","hai_long"]], on="ma_sv",
validate="one_to_one")
ket_qua.to_csv("ket_qua_etl.csv", index=False, encoding="utf-8-sig")
```

Lưu ý: API_URL là biến cần điền URL thực; mẫu JSON giả định khóa "data". Nếu API khác cấu trúc, cần điều chỉnh bước đọc. Mã sinh viên nên chuẩn hóa khoảng trắng, kiểu chữ trước khi ghép; dữ liệu nhạy cảm cần được bảo vệ.

9. Prompt AI cho từng bước thực hiện

Prompt 1 – Khảo sát nguồn

> "Đóng vai Data Analyst. Hãy kiểm tra sơ đồ dữ liệu CSV điểm, JSON khảo sát và PDF quy định; liệt kê cột, khóa ghép, dữ liệu thiếu và rủi ro sai lệch. Không tự bịa dữ liệu."

Prompt 2 – Sinh mã làm sạch

> "Viết Python Pandas chạy trên Colab: đọc CSV, gọi API bằng Requests, đọc PDF bằng pdfplumber, chuẩn hóa ma_sv, điểm [0,10], hài lòng [1,5], kiểm tra trùng và ghi log trước/sau."

Prompt 3 – Tích hợp & phân tích

> "Viết mã merge 1–1 theo ma_sv, báo cáo mã không ghép được, xuất CSV và tạo bảng/biểu đồ mô tả điểm và hài lòng. Chỉ kết luận theo dữ liệu thực có."

Prompt 4 – So sánh phản biện

> "So sánh Pandas, NiFi, Airflow theo chi phí triển khai, học tập, trích xuất dữ liệu, biến đổi, lịch chạy và giám sát; nêu rõ vì sao Pandas phù hợp nhóm nghiên cứu và giới hạn khi mở rộng."

10. Kiểm chứng và kết luận

- Đối soát số dòng ở từng bước; không xóa trùng theo mã nếu các dòng thể hiện các lần khảo sát hợp lệ khác nhau.
- Chỉ chấp nhận ngưỡng từ PDF sau khi kiểm tra trang, ngày ban hành và phạm vi áp dụng.
- Đọc lại tệp xuất; kiểm tra duy nhất ma_sv, tỷ lệ ghép, giá trị ngoài miền và kết luận bằng số liệu.

Kết luận lựa chọn: Với bài nghiên cứu kết hợp dữ liệu điểm (CSV), khảo sát (API) và quy định (PDF), Pandas là phương án cốt lõi vì dễ làm sạch, ghép bảng và tạo thống kê trong notebook. NiFi đáng cân nhắc nếu thu thập liên tục ở nhiều đầu mối; Airflow phù hợp khi lịch ETL, retry và giám sát trở thành yêu cầu chính.

Tài liệu tham khảo chính thức (truy cập 09/11/2026)

[1] pandas – User Guide: https://pandas.pydata.org/docs/user_guide/

[2] pandas – merge: https://pandas.pydata.org/docs/reference/api/pandas.merge.html

[3] Apache NiFi – Overview: https://nifi.apache.org/nifi-docs/overview.html

[4] Apache Airflow – Documentation: https://airflow.apache.org/docs/apache-airflow/

[5] pdfplumber – README: https://github.com/jsvine/pdfplumber